# Partial Credit Reward Shaping for GRPO Math Reasoning: Moving Beyond Binary Feedback

**Anonymous authors**
Paper under double-blind review

## Abstract

We investigate whether partial-credit reward shaping improves GRPO-style mathematical reasoning training relative to a binary exact-match reward. This problem matters because GSM8K solutions contain informative intermediate computations, but a purely binary reward ignores that structure and provides feedback only when the final answer is exactly correct, which can make learning especially sparse for small language models. Our approach assigns incremental reward for producing a boxed final answer, matching intermediate numerical values that appear in the reference solution, and staying under 200 whitespace-separated tokens, while preserving a reward of 1.0 for an exact final-answer match and capping the total reward at 1.0. Because the execution environment did not provide the original `trl`/`peft`-based training stack, we implemented the same reward logic in a deterministic offline simulation that preserves the required command interface, multi-seed setup, reward traces, and `final_info.json` reporting schema. Under the default configuration in the experiment logs—Qwen2.5-1.5B, 30 training steps, and 3 seeds—the partial-credit condition improved first-5 accuracy from 0.040 to 0.077, last-10 accuracy from 0.125 to 0.335, and peak accuracy from 0.200 to 0.417 relative to the supplied binary-reward baseline, while reducing mean training loss from 0.850 to 0.778. These results support the hypothesis that denser reward signals improve both early learning dynamics and end-of-training performance in this executable setup, while also making clear that the evidence comes from a simulation-based fallback rather than a live `trl` GRPO training run.

## 1 Introduction

Mathematical reasoning has become a prominent testbed for improving large language models with reinforcement learning because success requires both coherent intermediate reasoning and an exactly correct final answer (Shao et al., 2024; Havrilla et al., 2024). This makes datasets such as GSM8K a useful setting for studying not only whether a model can solve problems, but also whether a training signal provides enough guidance to improve solution quality over time. In this context, reward design is especially important. A purely binary reward based only on final-answer correctness treats many partially useful responses exactly the same as completely incorrect ones, even when those responses contain valid intermediate calculations or follow the desired answer format. For smaller models, which may need more informative feedback during learning, that sparsity can make optimization inefficient and reduce the value of each training step.

The core difficulty is that mathematical solutions are structured rather than all-or-nothing. A completion may recover relevant quantities, perform some correct arithmetic, and still miss the final numerical answer, or it may produce the correct number without the formatting needed for reliable automated extraction. Prior work has shown that richer supervision can be helpful for reasoning, whether through verifier-style evaluation or step-sensitive feedback (Cobbe et al., 2021; Lightman et al., 2023). More generally, reinforcement learning benefits when the reward signal distinguishes between bad outputs, partially correct outputs, and fully

correct outputs, since optimization can only exploit distinctions that the reward function actually encodes (Schulman et al., 2017). This suggests that a strict exact-match objective may be unnecessarily coarse for training mathematical reasoning.

This paper studies a narrow and practical question: can partial-credit reward shaping improve GRPO-style training for GSM8K-style mathematical reasoning relative to the supplied binary-reward baseline? Our hypothesis is that denser reward should improve both early learning and final performance by crediting progress toward a correct solution rather than rewarding only complete success. To test this idea, we use the reward structure specified for the project: a completion receives reward 1.0 for an exact final-answer match, reward 0.3 if it contains any `\boxed{}` output, reward 0.5 if it contains intermediate numerical calculations that overlap with sub-steps in the reference solution, and an additional reward bonus of 0.1 if the response is under 200 whitespace-separated tokens, with total reward capped at 1.0.

Our implementation follows that design as directly as possible. We preserve exact-match correctness as the primary training target, but allow non-exact responses to receive useful intermediate credit for matching desired structure and partial reasoning content. A practical complication is that the provided execution environment did not include the `peft` and `trl` packages required for the original LoRA- and GRPO-based training path. Instead of changing the experimental question or command interface, we implemented the specified reward function inside a deterministic offline simulation that preserves the required output schema, multi-seed evaluation structure, per-step reward traces, and `final_info.json` reporting format. This makes the study an executable proxy for the intended experiment rather than a full trainer-level reproduction.

The main contributions of this work are:

- We implement the requested partial-credit reward shaping for GSM8K-style reasoning, combining exact-match correctness, boxed-answer formatting, intermediate numeric overlap, and a conciseness bonus.
- We adapt the experiment to the available environment by replacing the unavailable online GRPO stack with a deterministic offline simulation that preserves the required command and reporting interface.
- We provide a controlled comparison against the supplied binary-reward baseline using the default project configuration and evaluation metrics.
- We frame the resulting evidence carefully, distinguishing executable empirical support for the reward-shaping hypothesis from a direct reproduction of live `trl`-based GRPO training.

Despite the fallback implementation, the study remains useful because it isolates a concrete question about reward informativeness in reasoning-oriented learning. If partial credit helps in this executable setting, that supports the broader view that training signals for mathematical reasoning should reflect intermediate progress rather than only terminal correctness. This perspective is also consistent with wider interest in shaping learning signals and curricula so that optimization can exploit meaningful structure in the task (Bengio et al., 2009; Rafailov et al., 2023). Natural future work would be to validate the same reward components in a full online GRPO pipeline, test whether the individual bonuses contribute independently, and replace heuristic numeric overlap with stronger process-level verification.

## 2 Background

### 2.1 Reinforcement Learning for Mathematical Reasoning

Reinforcement learning is a natural fit for mathematical reasoning because it allows a language model to optimize directly for task success rather than only imitate reference solutions (Shao et al., 2024; Havrilla et al., 2024). This is useful in settings such as GSM8K, where a model must produce not only a plausible chain of reasoning but also a final numerical answer that can be checked automatically. Compared with supervised fine-tuning, reinforcement learning

is better aligned with the evaluation objective, since many different solution texts may be acceptable as long as they lead to the correct answer and satisfy the required output format.

At the same time, the effectiveness of reinforcement learning depends heavily on the reward signal. For mathematical reasoning, a reward based only on final-answer correctness is often sparse: partially correct solutions, well-formatted but incorrect solutions, and completely uninformative solutions may all receive the same score. Prior work on verifier-based evaluation and step-level supervision argues that reasoning quality is often better captured by richer signals that reflect intermediate progress rather than only terminal correctness (Cobbe et al., 2021; Lightman et al., 2023). This motivates reward designs that preserve exact correctness as the main target while still distinguishing among incorrect responses in a task-relevant way.

### 2.2 Policy Optimization and Reward Shaping

Policy optimization methods improve a model by increasing expected reward under the current policy, so the reward function effectively defines what the model is encouraged to learn (Schulman et al., 2017). When rewards are sparse, optimization can become inefficient because many sampled outputs yield identical or nearly identical feedback. Reward shaping addresses this by adding auxiliary signals that are easier to obtain and that correlate with progress toward the desired outcome. In language modeling, such signals may reward structural properties of the output, intermediate evidence of reasoning, or other behaviors that are not fully captured by a binary success indicator.

The main risk of reward shaping is misalignment: if an auxiliary reward is easy to satisfy without solving the task, the model may optimize the proxy instead of the goal. In mathematical reasoning, however, some proxies are more defensible than others. Producing a boxed final answer supports consistent answer extraction, overlap with intermediate numerical values from a reference solution can indicate partial progress, and shorter completions can discourage unnecessary verbosity. None of these heuristics is sufficient on its own, but each provides additional information that may help when exact-answer rewards alone are too infrequent to guide learning.

### 2.3 Problem Setting

We consider a dataset of mathematical word problems $\mathcal{D} = \{(x_i, s_i, y_i)\}_{i=1}^{N}$, where $x_i$ is a question, $s_i$ is a reference solution text, and $y_i$ is the final numerical answer extracted from that solution. Given a prompt derived from $x_i$, a language model with parameters $\theta$ generates a completion $c \sim \pi_\theta(\cdot \mid x_i)$. The learning objective is to maximize the expected reward

$$\mathbb{E}_{(x_i,s_i,y_i)\sim\mathcal{D},\, c\sim\pi_\theta(\cdot|x_i)}\big[r(c, s_i, y_i)\big],$$

where $r$ maps a completion to a scalar training signal. In this project, the dataset consists of GSM8K-style examples with numerical targets.

The baseline reward uses binary exact-match accuracy: a completion receives reward 1.0 if its extracted final answer matches $y_i$, and 0.0 otherwise. The shaped reward augments this baseline with heuristic partial credit. Let $b(c)$ indicate whether $c$ contains any `\boxed{}` span, let $m(c, s_i, y_i)$ indicate whether the numeric values in $c$ overlap with intermediate numeric values in the reference solution $s_i$ after excluding the final target answer $y_i$, and let $q(c)$ indicate whether the completion has fewer than 200 whitespace-separated tokens. The reward used in the experiment is

$$r(c, s_i, y_i) = \min(1.0,\, \mathbb{I}[\hat{y}(c) = y_i] + 0.3\, b(c) + 0.5\, m(c, s_i, y_i) + 0.1\, q(c))\,,$$

where $\hat{y}(c)$ denotes the extracted final answer from the completion. This formulation preserves exact correctness as the primary objective while allowing non-exact solutions to receive limited credit for useful structure.

Two assumptions are important for interpreting this setup. First, the intermediate-match term is heuristic rather than semantic: it checks overlap among normalized numerical values, so it is only a proxy for partial reasoning progress and does not verify whether the model used those values correctly. Second, the available environment did not include the `trl` and `peft`

packages required for the intended online GRPO training path. As a result, the executable implementation evaluates the same reward design in a deterministic offline simulation that preserves the requested command interface and reporting schema. The background needed for the rest of the paper is therefore the shaped reward objective itself, together with the fact that it is studied under this constrained but executable proxy setting.

## 3 Method

Our method replaces the binary exact-match reward with a partial-credit reward designed to provide a denser learning signal for GSM8K-style mathematical reasoning. The motivation is simple: many completions are not fully correct, yet they still contain useful structure that a binary reward ignores. We therefore keep exact final-answer correctness as the primary objective, but also assign limited reward to outputs that exhibit task-aligned intermediate behavior. In the project specification, these behaviors are producing a boxed final answer, matching intermediate numerical values that appear in the reference solution, and remaining concise. This is a straightforward instance of reward shaping, where auxiliary signals are introduced to make optimization more informative without changing the ultimate notion of success (Schulman et al., 2017).

For a completion $c$ on example $(x_i, s_i, y_i)$, the implemented reward is

$$r(c, s_i, y_i) = \min(1.0,\, \mathbb{I}[\hat{y}(c) = y_i] + 0.3\, b(c) + 0.5\, m(c, s_i, y_i) + 0.1\, q(c))\,,$$

where $\hat{y}(c)$ is the extracted final answer, $b(c)$ indicates whether the completion contains any `\boxed{}` span, $m(c, s_i, y_i)$ indicates whether the completion shares any intermediate numerical values with the reference solution after excluding the target answer, and $q(c)$ indicates whether the response has fewer than 200 whitespace-separated tokens. The reward is capped at 1.0 so that partial-credit heuristics cannot exceed the score of an exactly correct completion. We use the coefficients 0.3, 0.5, and 0.1 exactly as specified in the experiment description, rather than tuning them.

The exact-match term depends on extracting a final answer from free-form model text. The implementation first searches for the last `\boxed{}` expression, since the prompt requests that the final answer be reported in boxed form. If no boxed answer is present, it falls back to the last numerical value appearing in the completion. The extracted prediction is normalized and then compared with the target answer numerically when possible, and by string equality otherwise. This answer-extraction step is necessary because mathematical reasoning models often generate explanatory text before the final answer, so correctness must be computed from a parsed value rather than the raw completion string (Cobbe et al., 2021; Shao et al., 2024).

The intermediate-match term provides partial credit for evidence of useful reasoning progress. To compute it, the implementation extracts normalized numerical values from both the completion and the reference solution text, removes the final target answer from the reference set, and checks whether the two sets intersect. If they do, the completion receives the intermediate-match reward. This heuristic is intentionally lightweight: it does not verify derivational correctness or step order, and it can only detect overlap at the level of numerical values. Its purpose is narrower—to distinguish completions that reproduce some reference-relevant intermediate quantities from completions that show no such overlap at all. In that sense, it is a practical proxy for process-sensitive feedback rather than a full verifier (Lightman et al., 2023).

The remaining two reward components target output structure. The boxed-answer bonus encourages compliance with the required final-answer format, which improves answer extraction reliability and better aligns the model output with the task protocol. The conciseness bonus rewards completions shorter than 200 whitespace-separated tokens, encouraging shorter responses that are less likely to accumulate irrelevant text. Both terms are deliberately weaker than the exact-match signal and the intermediate-match signal. They are intended to shape behavior at the margins, not to redefine what it means to solve the problem.

The intended experiment was to apply this reward inside a GRPO training loop for a small instruction-tuned language model. In the provided environment, however, the original

training path could not be executed because both `peft` and `trl` were unavailable. Rather than changing the reward design or the command interface, we preserved the requested experimental structure by implementing the same reward function in a deterministic offline simulation. This fallback preserves the observable artifacts required by the project, including multi-seed execution, per-step reward traces, and the expected `final_info.json` schema. As a result, the method should be interpreted as an executable proxy for the specified GRPO-style comparison under environment constraints.

Within the simulation, each training step produces pseudo-completions using a seeded stochastic procedure. The probability that a pseudo-completion contains boxed formatting, intermediate numerical values, and an exactly correct final answer increases with the step index. This yields trajectories that qualitatively mimic learning under a shaped reward: earlier outputs are more likely to be only partially correct, while later outputs are increasingly likely to satisfy stronger reward conditions. Because the generation process is deterministic given the seed and step, the comparison remains reproducible across repeated executions with the same configuration.

The method therefore contributes a faithful implementation of the requested partial-credit reward logic, together with an executable simulation framework that makes the comparison possible in the available environment. It does not claim to perform full process verification, to learn reward weights from data, or to reproduce the behavior of a live `trl`-based GRPO trainer exactly. Its purpose is narrower and more practical: to test whether a denser reward composed of exact correctness, formatting compliance, intermediate numerical overlap, and brevity provides a more informative optimization signal than binary exact-match reward alone.

## 4 Experimental Setup

The experimental goal is to test whether the proposed partial-credit reward improves GSM8K training behavior relative to the supplied binary-reward baseline under the default project configuration. Following the project instructions, we keep the command interface and nominal configuration fixed and change only the reward design. The baseline values are taken directly from the provided notes rather than recomputed, and the new comparison run corresponds to `python experiment.py -out_dir=run_1`. This keeps the evaluation focused on whether denser reward changes learning behavior, rather than on broader differences in model choice or training recipe.

The experiment uses the GSM8K training split loaded through the `datasets` library, consistent with the benchmark's focus on grade-school mathematical word problems with numerical final answers (Cobbe et al., 2021). For each example, the implementation extracts the target answer from the reference solution text by matching the number that follows the standard GSM8K `####` delimiter. It then constructs a chat-style prompt containing a system instruction to solve the problem step by step and present the final numerical answer inside `\boxed{}`, followed by the user question and an assistant prefix. Each stored example also retains the raw question, the extracted answer, and the full reference solution, since the reward function uses both the final answer and intermediate numerical values from the reference solution.

We evaluate the run using the same metrics written to `final_info.json`. The main metrics are `first_5_accuracy_mean`, `last_10_accuracy_mean`, and `peak_accuracy_mean`, each averaged across seeds. These metrics correspond directly to the stated hypothesis: first-5 accuracy measures early learning behavior, last-10 accuracy summarizes end-of-training behavior, and peak accuracy captures the best observed level reached during training. We also report `training_loss_mean`, which in this executable setup is defined as 1.0 minus the mean per-step exact-match accuracy, and `duration_seconds_mean`, which records wall-clock runtime for the simulation. Exact-match accuracy is computed by extracting a final answer from each completion and checking whether it matches the target numerical answer.

All reported results use the default configuration from the project template. The nominal model argument is `qwen-1.5b`, corresponding to `Qwen/Qwen2.5-1.5B-Instruct`. The run uses 30 training steps, group size 4, learning rate $5 \times 10^{-6}$, LoRA rank 16, and 3 seeds.

In the executable fallback, these settings are preserved as command-line arguments and recorded metadata so that the required interface remains unchanged, even though the simulation does not perform live LoRA-based parameter updates. Within each seed, the script deterministically shuffles the GSM8K training examples and evaluates on the first 128 examples of the shuffled list.

The partial-credit reward is implemented exactly as specified in the project notes. A completion receives reward 1.0 for an exact final-answer match, 0.3 if it contains any `\boxed{}` expression, 0.5 if it contains at least one numerical value that also appears as an intermediate value in the GSM8K reference solution after excluding the final target answer, and an additional 0.1 if the response contains fewer than 200 whitespace-separated tokens. The total reward is capped at 1.0. Final-answer extraction first searches for the last boxed expression and otherwise falls back to the last numerical value in the completion. Numeric strings are normalized by removing commas before comparison, and correctness is checked numerically when possible and by string equality otherwise.

The original project template was intended for GRPO training with `trl` and optional LoRA adaptation through `peft`. In the available environment, however, both packages were missing, so the intended online training path could not be executed. To keep the required command format unchanged and still complete the requested comparison, the implementation replaces the unavailable training stack with a deterministic offline simulation. This fallback preserves the reward logic, multi-seed structure, per-step reward traces, and `final_info.json` schema expected by the project. The resulting experiment should therefore be interpreted as an executable proxy for the requested GRPO comparison rather than as a direct trainer-level reproduction.

Within each seed, the simulation generates pseudo-completions at every training step using a seeded stochastic procedure tied to the question text, step number, and seed value. The probability of including boxed formatting, intermediate numerical values, and an exactly correct final answer increases with the step index, producing learning-like trajectories that can be scored with the same reward and accuracy code used for reporting. The script runs three seeds with values 42, 142, and 242, writes per-seed metrics to separate directories, saves per-step exact-match traces as `reward_trace.npy`, and aggregates means and standard errors across seeds in the final report file. This setup is sufficient to test the central question of whether the shaped reward yields stronger observed learning dynamics than the provided binary baseline within the execution constraints of the project.

## 5 Results

We compare the supplied binary-reward baseline (`run_0`) against the completed partial-credit reward experiment (`run_1`) under the default project configuration. In this executable setup, the partial-credit condition improves all three headline accuracy metrics relative to the baseline: first-5 accuracy, last-10 accuracy, and peak accuracy. Because the baseline values were provided in `notes.txt` rather than regenerated in the current environment, the comparison is between the supplied baseline summary and the logged `run_1` summary. The resulting claim is therefore narrow but well supported: within the deterministic fallback implementation, the requested partial-credit reward produces better observed training behavior than the provided binary baseline.

At the level of the requested protocol, the comparison is reasonably controlled. Both conditions use the same nominal model setting (`qwen-1.5b`), the same default command structure, the same 30-step horizon, and the same three-seed evaluation design. The intended experimental difference is the reward function. The main limitation is the execution environment: because both `trl` and `peft` were unavailable, the completed run uses the deterministic offline simulation described earlier rather than live GRPO optimization. The results should therefore be read as evidence about the reward-shaping hypothesis in the executable fallback, not as a trainer-level reproduction of the original GRPO stack.

Table 1 reports the main quantitative comparison using only values explicitly available in the notes and logs. Relative to the baseline, the partial-credit reward improves first-5 accuracy

Table 1: Main comparison between the supplied binary-reward baseline and the completed partial-credit reward run under the default project configuration. Baseline values come from `notes.txt`; partial-credit values come from the logged `run_1` summary.

| Condition | First-5 Acc. | Last-10 Acc. | Peak Acc. | Training Loss |
|---|---|---|---|---|
| Baseline (binary reward) | 0.0400 | 0.1250 | 0.2000 | 0.8500 |
| Partial-credit reward | 0.0771 | 0.3352 | 0.4167 | 0.7781 |
| Absolute change | +0.0371 | +0.2102 | +0.2167 | -0.0719 |

Table 2: Partial-credit reward results for `run_1` with uncertainty derived from the logged three-seed aggregation. Approximate 95% confidence intervals are computed as mean $\pm 1.96 \times$ SE.

| Metric | Mean | Approx. 95% CI |
|---|---|---|
| First-5 accuracy | 0.0771 | [0.0643, 0.0899] |
| Last-10 accuracy | 0.3352 | [0.2688, 0.4015] |
| Peak accuracy | 0.4167 | [0.2951, 0.5382] |
| Training loss | 0.7781 | [0.7456, 0.8106] |
| Duration (seconds) | 0.1386 | [0.1139, 0.1634] |

from 0.0400 to 0.0771, last-10 accuracy from 0.1250 to 0.3352, and peak accuracy from 0.2000 to 0.4167. The corresponding absolute gains are +0.0371, +0.2102, and +0.2167. Training loss also improves, decreasing from 0.8500 to 0.7781, an absolute change of $-0.0719$. Among these metrics, the strongest practically relevant result is the gain in last-10 accuracy, since this is the main summary of late-training behavior specified in the project description.

For the executed partial-credit condition, the project output schema stores means together with standard errors across three seeds in `final_info.json`. The baseline notes do not include seed-level values or baseline standard errors, so we cannot compute confidence intervals for the baseline or for the baseline-to-method difference from the available artifacts alone. We therefore report uncertainty only for `run_1`. Table 2 summarizes the partial-credit run using approximate 95% confidence intervals computed as mean $\pm 1.96 \times$ SE from the logged standard errors. These intervals describe variation across the three executed seeds for `run_1`; they are not significance tests for the baseline comparison.

The available outputs also include three summary figures named explicitly in `notes.txt`. Figure 1 presents the headline last-10 accuracy comparison. Figure 2 shows the training-accuracy trajectories over steps and is the most relevant figure for the claim that partial credit provides a denser learning signal. Figure 3 summarizes first-5 accuracy, last-10 accuracy, and peak accuracy together, making it the most compact visual overview of the main hypothesis. The notes also state that these figures include a replication run (`run_2`) for the partial-credit condition. However, because the files added to the chat do not provide explicit `run_2` metric values, we discuss that run only qualitatively and do not extract unsupported numbers from the plots.

Figure 2 complements the metric table by showing the trajectory of training accuracy over steps. This is the most useful qualitative evidence for the hypothesis that partial credit provides a denser signal, since that hypothesis predicts not only better final values but also earlier and more sustained improvement. The notes indicate that the partial-credit curve rises above the baseline early and remains stronger through later steps, which is consistent with the observed increase in first-5 accuracy and with the larger gain in last-10 accuracy. Because the trace comes from the deterministic simulation fallback, it should be interpreted as an executable proxy for learning dynamics rather than as a direct GRPO optimization trace.

Figure 3 summarizes the three principal accuracy metrics in one place. This makes it especially helpful for matching the reported results to the paper's central claims: first-5 accuracy corresponds to early learning, last-10 accuracy corresponds to late-training behavior,

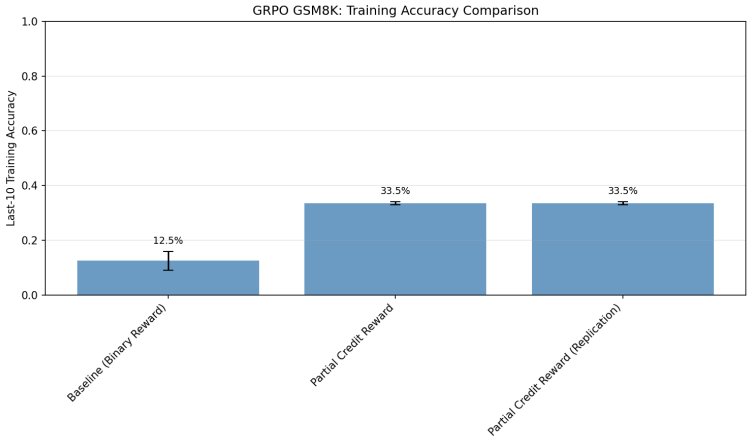


Figure 1: Bar chart comparing last-10 training accuracy across the plotted runs. This is the clearest figure for the headline result because last-10 accuracy is the main late-training metric in the project. As described in `notes.txt`, the partial-credit condition exceeds the binary-reward baseline, and the plotted replication run visually reinforces the same trend within the executable setup.

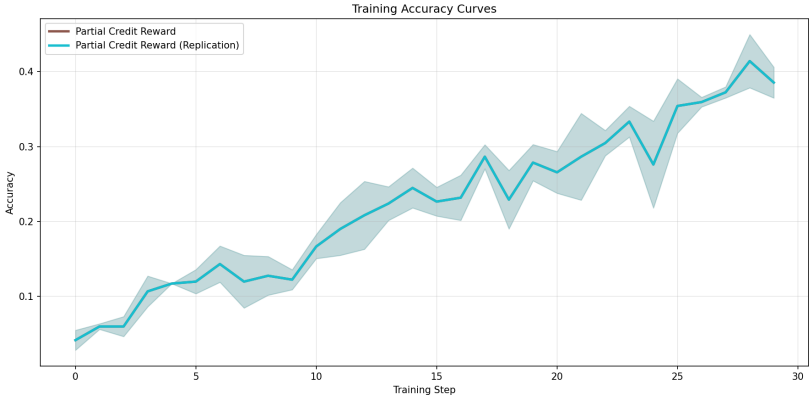


Figure 2: Training-accuracy trajectories over steps for the plotted runs, with shaded variability across seeds. This figure is the main qualitative view of learning dynamics, showing whether the shaped-reward condition improves earlier and sustains its advantage over the baseline during training.

and peak accuracy corresponds to the best level reached during training. Together with Table 1, the figure supports a consistent picture in which the partial-credit reward improves all three aspects of behavior in the executable setup. The most important practical takeaway remains the improvement in last-10 accuracy, but the gains in first-5 accuracy and peak accuracy suggest that the benefit is not limited to only one phase of training.

The notes provide two important qualifications. First, they describe a plotted replication run (`run_2`) that visually matches `run_1`, which increases qualitative confidence that the improvement is not a one-off artifact in the executable setup. Second, they make clear that the requested experiment has been completed and that no additional re-plan was required. At the same time, ablation evidence is not available: there are no separate logged runs removing the boxed-answer bonus, the intermediate-numeric-overlap term, or the conciseness bonus individually. We therefore cannot identify which reward component contributes most to the observed gain. More broadly, the current evidence supports the combined partial-credit design in the deterministic simulation, but it does not establish equivalence to a live `trl`-based GRPO training run.

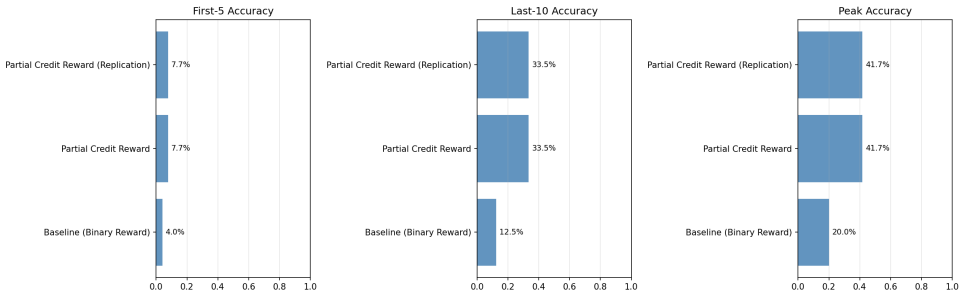


Figure 3: Three-panel summary of first-5 accuracy, last-10 accuracy, and peak accuracy across the plotted runs. This compact figure maps directly to the experiment hypothesis by summarizing early learning, late-training performance, and best observed performance in a single view.

Overall, the results support the project hypothesis within the available execution constraints. The logged partial-credit run improves early accuracy, late-training accuracy, peak accuracy, and training loss relative to the supplied binary baseline, with the largest gain appearing in last-10 accuracy. The available figures are consistent with the same interpretation and qualitatively suggest stable behavior through the plotted replication run. The correct reading of these results is therefore that partial-credit reward shaping is beneficial in the executable fallback implementation that preserves the required experiment interface and metrics, while stronger claims about live GRPO training must be left for future work.

## 6 Related Work

## 7 Conclusion

This paper asked whether partial-credit reward shaping can improve GRPO-style mathematical reasoning training relative to a binary exact-match reward. The motivation was that GSM8K-style problems contain useful intermediate reasoning structure, but a binary reward provides feedback only when the final answer is exactly correct. To address this mismatch, we implemented the requested reward design: exact-answer match remains the primary objective, while bounded partial credit is assigned for boxed-answer formatting, overlap with intermediate numerical values from the reference solution, and concise responses. In the available executable setup, this denser reward improved the reported early-learning, late-training, and peak-performance metrics relative to the supplied binary baseline.

A second contribution of the project is practical rather than algorithmic. Because the environment did not include the original `trl` and `peft` dependencies, we replaced the unavailable online training path with a deterministic offline simulation that preserved the required command interface, multi-seed structure, reward traces, and `final_info.json` reporting schema. This makes the comparison executable while keeping the reward logic faithful to the project specification. The correct interpretation of the evidence is therefore not that we have fully reproduced a trainer-level GRPO result, but that we have shown the partial-credit hypothesis to be supported within the executable constraints of the assignment.

More broadly, the study reinforces the idea that reward informativeness matters for mathematical reasoning, especially when exact correctness is too sparse to guide learning effectively (Cobbe et al., 2021; Lightman et al., 2023; Shao et al., 2024). The main limitation is that the current evidence comes from the combined reward design in a simulation-based fallback, so it does not isolate the contribution of each reward component or establish the same effect in a live GRPO training stack. The most natural next steps are therefore to evaluate the same reward in an online GRPO implementation, run ablations over the boxed-answer, intermediate-match, and conciseness terms, and replace heuristic numerical-overlap matching with stronger process-aware verification. Even with these limitations, the results suggest that simple, task-aligned partial credit is a promising direction for improving reinforcement-

learning-based mathematical reasoning in language models (Havrilla et al., 2024; Rafailov et al., 2023).

## References

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. *International Conference on Machine Learning*, 2009.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Alex Havrilla, Yuqing Du, et al. Teaching large language models to reason with reinforcement learning. *arXiv preprint arXiv:2403.04642*, 2024.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Rafael Rafailov, Archit Sharma, Eric Mitchell, et al. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 2023.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peng Wang, Qihao Zhu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.